Soal Modul 2 Strukdat 2025

- Soal 1 **(encryption....encryption....)**

Kamu merupakan suatu agen dalam suatu organisasi rahasia, yang dimana sekarang kamu sedang menjalankan tugas yang sangat berbahaya untuk memulihkan kondisi negaramu yang sedang terpuruk. Saat ini, kamu sedang menunggu suatu kode angka yang harus dimanipulasi dengan pola tertentu. Kode-kode ini tersimpan dalam sebuah **singly linked list** dan mengandung informasi vital untuk membuka brankas rahasia.

Untuk mengakses informasi yang tersembunyi, kamu harus melakukan operasi khusus pada daftar kode tersebut:

- Memilih dua posisi a dan b dalam daftar kode (di mana a ≤ b)
- Membalik urutan kode dari posisi a hingga b
- Sekaligus menukar posisi rangkaian kode yang telah dibalik tersebut dengan rangkaian kode yang sama panjangnya di bagian akhir daftar

Input Format

Program menerima input sebagai berikut:

- Baris pertama berisi **n**, jumlah node dalam list.
- Baris kedua berisi **n** bilangan yang menjadi node dari list.
- Baris ketiga berisi range, **a dan b.**

Constraints

- 0 < n < 10000, n harus bilangan ganjil (n mod 2 == 1)
- **Indeks dimulai dari 1** dan tidak melebihi panjang list.
- 1 ≤ a ≤ b ≤ n
- Panjang range tidak boleh melebihi n / 2

Output Format

- Jika n tidak memenuhi constraint, print “whuh”
- Jika range tidak memenuhi constraint, print “-w- fix your range”
- Format output list:

```
:3 nodes = node -> node -> node  -> node -> node -> node -> node
```

Sample Input 1

```
7
1 2 3 4 5 6 7
1 2
```

Sample Output 1

```
:3 nodes = 7 -> 6 -> 3 -> 4 -> 5 -> 2 -> 1
```

Sample Input 2

```
15
1 4 7 10 13 16 19 22 25 28 31 34 37 40 45
2 5
```

Sample Output 2

```
:3 nodes = 1 -> 40 -> 37 -> 34 -> 31 -> 16 -> 19 -> 22 -> 25 -> 28 -> 13 -> 10 -> 7 -> 4 -> 45
```

Sample Input 3

```
-1
```

Sample Output 3

```
whuh
```

Sample Input 4

```
7
1 2 3 4 5 6 7
1 5
```

Sample Output 4

```
-w- fix your range
```

Code Snippet

```
#include <iostream>
using namespace std;

class SListNode {
public:
    int data;
```

```
    SListNode* next;

    SListNode(int value) {
        data = value;
        next = nullptr;
    }
};

class SinglyList {
private:
    SListNode* head;
    unsigned size;

    public:
    SinglyList() {
        head = nullptr;
        size = 0;
    }

    bool isEmpty() {
        // ...
    }

    void pushBack(int value) {
        // ...
    }

    void printList() {
        // ...
    }

    void solution(int a, int b) {
        // ...
    }
};

int main() {
    // ...
    return 0;
}
```

- Soal 2 **(Kavi Nugas)**

Kavi adalah seorang mahasiswa semester 2 jurusan Teknologi Informasi. Suatu hari, dosennya memberikan sebuah tugas yaitu **menggabungkan dua daftar nilai mahasiswa** dari dua kelas berbeda.

Masing-masing kelas sudah menyusun nilai mahasiswanya secara **urut dari yang terkecil ke yang terbesar**. Tapi karena data dari dua kelas itu dikumpulkan dalam bentuk **linked list**, Kavi tidak bisa langsung pakai array biasa.

Tugas Kavi adalah:

1. Menyimpan nilai-nilai mahasiswa ke dalam dua linked list terurut (kelas A dan kelas B).
2. Menggabungkan kedua linked list terurut menjadi satu linked list yang tetap terurut.
3. Menampilkan hasil gabungan kedua linked list.

Input Format

1. Pengguna diminta untuk memasukkan **jumlah elemen** yang ingin dimasukkan dalam masing-masing linked list (kelas A dan kelas B).
2. Kemudian, pengguna akan memasukkan **nilai-nilai** untuk kedua kelas tersebut, yang **harus terurut menaik**.

Constraint

1. Jangan membuat node baru saat menggabungkan.
2. Gunakan hanya manipulasi pointer (next) dari node-node yang sudah ada.
3. Jangan Gunakan STL

Output Format

Isi dari linked list **kelas A**.
Isi dari linked list **kelas B**.
Gabungan kedua linked list tersebut menjadi **satu linked list terurut**.

Sample Input 1

```
3
70 75 77
3
20 45 67
```

Sample Output 1

```
Kelas A: 70 75 77
Kelas B: 20 45 67
Nilai Gabungan: 20 45 67 70 75 77
```

Sample Input 2

```
3
70 75 77
3
71 72 73
```

Sample Output 2

```
Kelas A: 70 75 77
Kelas B: 71 72 73
Nilai Gabungan: 70 71 72 73 75 77
```

Sample Input 3

```
2
15 17
5
18 19 22 23 27
```

Sample Output 3

```
Kelas A: 15 17
Kelas B: 18 19 22 23 27
Hasil Gabungan: 15 17 18 19 22 23 27
```

Sample Input 4

```
2
1 2
0
```

Sample Output 4

```
Kelas A: 1 2
Kelas B:
Hasil Gabungan: 1 2
```

- Soal 3 **(Perjalanan TimeLoop Ludwigd: Mengatur Linimasa)**

Ludwigd adalah seorang penjelajah waktu yang terjebak dalam linimasa yang kacau. Untuk kembali ke masa kini, ia harus menyusun peristiwa-peristiwa penting dalam struktur **Circular Singly Linked List**. Setiap node mewakili sebuah peristiwa, dan karena waktu bersifat melingkar, node terakhir harus terhubung ke node pertama. Bantu Ludwigd menambahkan, menghapus, atau menyisipkan peristiwa di posisi tertentu agar ia dapat mengatur linimasa dan menemukan jalan pulang!

Input Format

Program menerima beberapa perintah dalam bentuk:

- `**TAMBAH_DEPAN <nilai>**`
- `**TAMBAH_BELAKANG <nilai>**`
- `**HAPUS <indeks>**`
- `**SISIP <indeks> <nilai>**`
- `**TAMPILKAN**`

Constraint

- Nilai adalah bilangan bulat positif (1 ≤ nilai ≤ 1000).
- Indeks dimulai dari 0 dan tidak melebihi panjang list.
- Jika list kosong, hanya perintah `**TAMBAH_DEPAN**` atau `**TAMBAH_BELAKANG**` yang valid.

Output Format

- Untuk setiap perintah `**TAMPILKAN**`, cetak seluruh isi list dipisahkan spasi, diakhiri newline.
- Jika operasi tidak valid, cetak `**Operasi gagal**`.

Code Snippet

```
#include <iostream>
using namespace std;
```

```cpp
// node untuk menyimpan data linimasa
class TimeNode {
public:

};

// class untuk mengatur linimasa circular
class TimeCircularList {
private:

public:
    TimeCircularList() : _head(nullptr), _size(0) {}

    bool isEmpty() {
        // Implementasikan
    }

    void tambahDepan(int value) {
        // Menambahkan node di depan
    }

    void tambahBelakang(int value) {
        // Menambahkan node di belakang
    }

    void hapus(int index) {
        // Menghapus node di indeks tertentu
    }

    void sisip(int index, int value) {
        // Menyisipkan node di indeks tertentu
    }

    void tampilkan() {
        // Mencetak seluruh isi list
    }
};

// Your code here
```

- Soal 4 **(TikTok Challenge: Viral Dance Carousel)**

Sebuah challenge dance viral di TikTok mengharuskan peserta menyusun gerakan dalam bentuk **Circular Doubly Linked List**. Setiap gerakan (node) terhubung ke gerakan sebelumnya (prev) dan berikutnya (next), membentuk lingkaran tak terputus. Peserta bisa menambah gerakan di depan/belakang, menghapus gerakan yang salah, atau menyisipkan gerakan baru di posisi tertentu. Challenge selesai jika carousel gerakan bisa ditelusuri maju (seperti video normal) dan mundur (seperti efek *rewind*) tanpa putus!

Input Format

Program menerima perintah dalam format:

- `**ADD_FRONT <nomor_gerakan>**`
- `**ADD_BACK <nomor_gerakan>**`
- `**DEL <indeks>**`
- `**INSERT <indeks> <nomor_gerakan>**`
- `**PLAY**` (traversal maju)
- `**REWIND**` (traversal mundur)
- `**DONE**`

Constraint

- Nomor gerakan: 1-1000 (misal: 1 = "Floss", 2 = "Renegade").
- Indeks ≥ 0 dan tidak melebihi panjang list.
- Operasi `**DEL**`/`**INSERT**` invalid jika list kosong.

Output Format

- Untuk `**PLAY**`/`**REWIND**`: Cetak nomor gerakan dipisahkan spasi.
- Jika operasi gagal: Cetak `**FAIL**`.
- Untuk `**DONE**`: Cetak `**Challenge selesai!**`.

Sample Input 0

```
ADD_BACK 3
ADD_FRONT 1
ADD_BACK 5
INSERT 1 2
DEL 3
PLAY
```

```
REWIND
DONE
```

Sample Output 0

```
1 2 3
3 2 1
Challenge selesai!
```

Code Snippet

```
#include <iostream>
using namespace std;

class DanceNode {
public:

};

class TikTokCarousel {
private:


public:
    TikTokCarousel() : _head(nullptr), _size(0) {}

    bool isEmpty() { /* Implementasi */ }
    void addFront(int value) { /* Tambah di depan */ }
    void addBack(int value) { /* Tambah di belakang */ }
    void del(int index) { /* Hapus gerakan */ }
    void insert(int index, int value) { /* Sisipkan gerakan */ }
    void play() { /* Traversal maju */ }
    void rewind() { /* Traversal mundur */ }
};

// Your Code Here
```

- Soal 5 **(Kavi Malas Antri)**

Di suatu pagi yang cerah, Kavi dan Vika sedang berjalan menyusuri pasar Turi di negeri Wakanda. Ia mendengar bahwa ada sebuah stand yang menjual Gundam Fix Platinum edisi terbatas. Sebagai kolektor sejati, ia tentu tak ingin melewatkannya. Namun, saat Kavi hendak mendekat ke stand itu, ia tertegun.

Antrean di depan stand itu tampak... *liar*.

Orang-orang menyela antrean sesuka hati, seperti tak ada aturan yang mengikat. Padahal antriannya sudah sesempit itu masih saja ada orang yang menyerobot.

"Waduh... ini sih bukan antrean, ini chaos," gumam Kavi.

Kavi pun memutuskan untuk menghindari keributan. Ia segera menghubungi temannya, Vika.

"Vik, tolong dong antriin. Aku lihat dari jauh aja," pinta Kavi sambil menepi ke bangku taman terdekat.

Vika mengangguk dengan senyum lesu. Ia tahu ini bukan urusan mudah. Dengan langkah mantap, ia masuk ke kerumunan antrean dan mencoba bertahan dari para penyusup Wakanda yang nyelonong tanpa rasa bersalah.

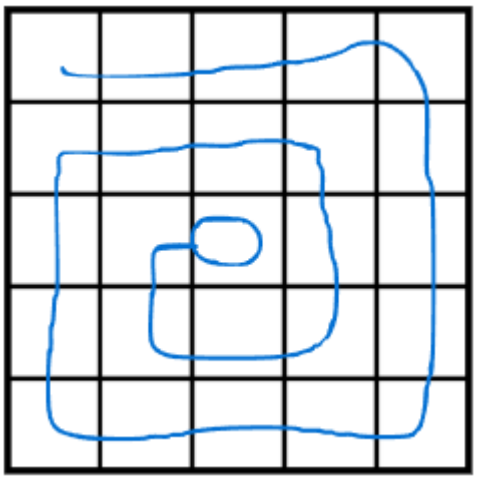
Sementara itu, Kavi dari kejauhan mengamati fenomena tersebut. Ia menghela nafas dan berkata dalam hati *"Beginilah wajah antrean di negeri tanpa norma. Kayak linked list tapi semua orang suka insert before sembarangan."*

Ia hanya bisa tersenym tipis sambil menertawakan mereka dalam hati. Di Wakanda, yang berdiri paling depan bukan yang datang paling dulu, tapi yang paling tebal mukanya.

Buatlah program untuk mensimulasikan antrean warga Wakanda di depan stand. Antrean dibentuk secara dinamis dengan aturan:

- Setiap orang memiliki **id unik** dan **muncul satu per satu. (dimulai dari 1)**
- Orang baru dapat memilih apakah ingin antre Setelah atau Sebelum **orang tertentu (Yang sudah datang : referensi by id).**
- **Stand bernomor 0**, dan posisinya **tetap di tengah antrean. Tidak boleh ada yang masuk sebelum 0.**
- Jika seseorang menyebut orang (ID) yang belum ada, program langsung berhenti (anggap input tidak valid).
- Setelah antrean lengkap, cetak dalam bentuk spiral outward dengan 0 sebagai pusat.

Ilustrasi Antrian :



Input Format

Program menerima beberapa perintah dalam bentuk:

```
N

ID POS IDREF
```

- N (Banyaknya Orang Yang Akan Masuk Antrian)
- ID (Unique Number Orang Yang Akan Antri)
- POS (Posisi orang tersebut terhadap orang dengan id = referensi. Contohnya :
    - A (After) : berarti orang dengan id akan ditempatkan setelah orang dengan id = referensi.
    - B (Before) : berarti orang dengan id akan ditempatkan sebelum orang dengan id = referensi.)
- IDREF (ID Orang Yang Sudah Antri)

Constraint

- Nilai adalah bilangan bulat positif (1 ≤ nilai ≤ 505000).
- Tidak boleh ada orang yang antri sebelum **Stand (0)**
- Tidak boleh menggunakan STL.

- Wajib menggunakan **OOP**

Output Format

- Mencetak Antrian Berbentuk **Spiral Outwards**
- Jika Salah Input Langsung End
- Rangenya Spiralnbisa pakai **(-)**

Sample input 1

5

1 A 0

2 A 1

3 A 2

4 A 3

5 A 4

Sample Output 1

-	-	-
1	0	5
2	3	4

Contoh Input 2 :

15

1 A 0

2 A 1

3 B 1

4 A 2

5 B 4

6 A 5

7 B 3

8 A 6

9 B 8

10 B 7

11 A 10

12 B 11

13 A 12

14 A 13

15 B 14

Contoh Output 2 :

-	-	-	-	-
1	3	7	11	-
2	10	0	14	-
5	12	13	15	-
6	9	8	4	-